
Kopie zapasowe i odzyskiwanie danych

Wydanie 0.0.1

Gal 1

18 cze 2026

Contents:

1	Dokumentacja bazy danych – Sklep elektroniczny	3
1.1	Wybór zagadnienia i identyfikacja danych	3
1.2	Opis procesów i towarzyszących im danych	3
1.3	Prototypowy plik JSON	4
1.4	Modelowanie koncektualne i logiczne	4
2	4. Definiowanie bazy danych i wprowadzanie danych do bazy	7
2.1	4.1. Definicja bazy danych w wariantach PostgreSQL i SQLite	7
2.2	4.2. Wybór mechanizmów wsadowego wprowadzania danych	9
2.3	4.3. Komentarz do procesu wprowadzania danych	9

Add your content using reStructuredText syntax. See the [reStructuredText](#) documentation for details.

Dokumentacja bazy danych – Sklep elektroniczny

Autorzy: Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

1.1 Wybór zagadnienia i identyfikacja danych

Tematem projektu jest podstawowa baza danych obsługująca sklep internetowy ze sprzętem elektronicznym. Baza danych zawiera następujące grupy danych:

- **Klienci:** unikalny identyfikator, imię, nazwisko, adres e-mail, numer telefonu oraz podstawowe dane adresowe do wysyłki: ulica, miasto, kod pocztowy.
- **Produkty:** unikalny identyfikator, nazwa sprzętu, cena jednostkowa oraz przypisana kategoria.
- **Kategorie:** identyfikator kategorii oraz jej nazwa, na przykład Laptopy lub Smartfony.
- **Zamówienia:** identyfikator zamówienia, data złożenia, status, na przykład nowe lub zrealizowane, oraz powiązanie z konkretnym klientem.
- **Szczegóły zamówienia:** pozycje przypisane do konkretnego zamówienia, określające jaki produkt został kupiony, w jakiej ilości oraz po jakiej cenie.

1.2 Opis procesów i towarzyszących im danych

1. **Zarządzanie produktami:** sklep oferuje asortyment przypisany do konkretnych kategorii, na przykład laptopy i smartfony. Każdy produkt ma przypisaną nazwę, cenę oraz identyfikator producenta.
2. **Zarządzanie użytkownikami:** klienci rejestrują się w systemie, podając podstawowe dane, takie jak imię, nazwisko i e-mail, oraz pojedynczy główny adres zamieszkania wykorzystywany do wysyłki.
3. **Realizacja zamówień:** zalogowany klient składa zamówienie na wybrane produkty. System tworzy nagłówek zamówienia zawierający datę i status oraz przypisuje do niego poszczególne produkty wraz z ich ilością.

1.3 Prototypowy plik JSON

Poniżej przedstawiono prototypowy plik w formacie JSON, zawierający jeden pełny dokument reprezentujący złożone zamówienie. Pozwala to zweryfikować kompletność gromadzonych i przetwarzanych informacji przed przejściem do modelowania konceptualnego.

```
{
  "id_zamowienia": 1,
  "data_zlozenia": "2026-05-14T10:15:30Z",
  "status": "Nowe",
  "klient": {
    "id_klienta": 4096,
    "imie": "Jan",
    "nazwisko": "Kowalski",
    "email": "student01@example.com",
    "telefon": "123456789",
    "ulica": "Armii Krajowej 78",
    "kod_pocztowy": "58-300",
    "miasto": "Walbrzych"
  },
  "pozycje": [
    {
      "id_produktu": 10543,
      "nazwa": "Laptop ASUS",
      "kategoria": "Laptopy",
      "ilosc": 1,
      "cena_historyczna": 6299.99
    }
  ]
}
```

1.3.1 Dobór typów danych

Tabela 1: Mapowanie typów danych dla SQLite i PostgreSQL

Atrybut danych	SQLite	PostgreSQL
Identyfikatory (id_...)	INTEGER	SERIAL lub INTEGER
Napisy krótkie (imie, nazwisko, miasto)	TEXT	VARCHAR(50)
Napisy długie (ulica, email, nazwa)	TEXT	VARCHAR(255)
Stałe ciągi znaków (telefon, kod_pocztowy)	TEXT	VARCHAR(15)
Cena (cena_bazowa, cena_historyczna)	FLOAT	NUMERIC(10,2)
Ilość (ilosc)	INTEGER	SMALLINT
Data i czas (data_zlozenia)	TEXT	TIMESTAMP

1.4 Modelowanie konceptualne i logiczne

1.4.1 Model koncepcyjny

1. Encje mocne

- **Klient:** klucz główny (id_klienta), imię, nazwisko, e-mail, telefon, ulica, miasto, kod pocztowy.
- **Produkt:** klucz główny (id_produktu), nazwa, cena_bazowa.

- **Kategoria:** klucz główny (`id_kategorii`), nazwa.
- **Zamówienie:** klucz główny (`id_zamowienia`), `data_zlozenia`, `status`.

2. Encje słabe

- **Szczegóły zamówienia:** klucz obcy (`id_zamowienia`), klucz obcy (`id_produktu`), `ilość`, `cena_historyczna`. Jej identyfikacja opiera się wyłącznie na kluczach obcych pochodzących z encji silnych.

3. Opis związków

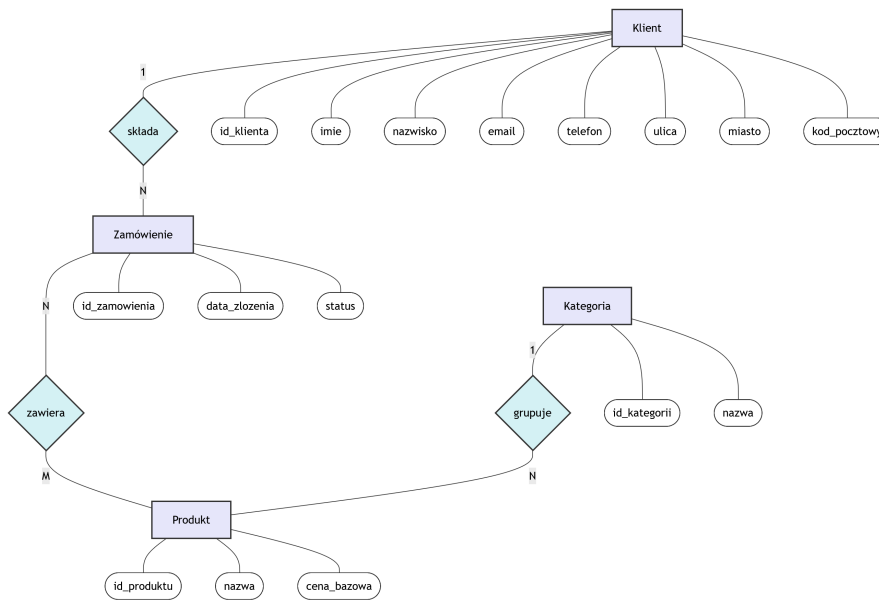
Zdefiniowano następujące relacje biznesowe między encjami:

- **Klient – Zamówienie (1:N):** klient składa zamówienie. Jeden klient może posiadać wiele zamówień, ale jedno konkretne zamówienie jest przypisane dokładnie do jednego klienta.
- **Kategoria – Produkt (1:N):** kategoria grupuje produkty. Kategoria może zawierać wiele produktów, ale dany produkt należy do jednej kategorii.
- **Zamówienie – Produkt (M:N):** zamówienie zawiera produkty. Pojedyncze zamówienie obejmuje wiele produktów, a dany produkt może pojawić się w wielu zamówieniach.

4. Związki niepoprawne

Wyeliminowano bezpośrednią relację między Klientem a Produktem, aby uniknąć pułapki połączeń. Bezpośrednie powiązanie gubi kontekst transakcji, na przykład datę i ilość zakupionego towaru, dlatego poprawna relacja musi zawsze przechodzić przez encję Zamówienie.

1.4.2 Model koncepcyjny – notacja Chena



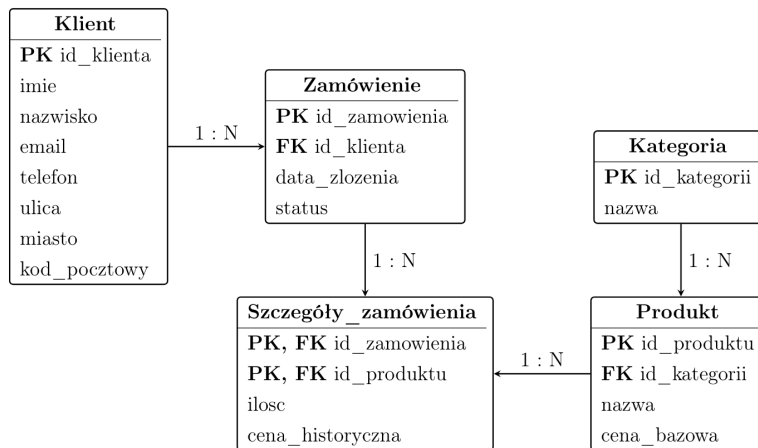
1.4.3 Model logiczny i normalizacja

Główne działania podjęte podczas normalizacji:

1. **I Postać Normalna (1NF):** dane adresowe klienta, takie jak ulica, miasto i kod pocztowy, zostały wyodrębnione jako pojedyncze pola.
2. **II Postać Normalna (2NF):** zlikwidowano relację wiele-do-wielu (N:M) pomiędzy Zamówieniem a Produktem, wstawiając encję asocjacyjną `Szczegóły_zamowienia`. Jej klucz główny składa się z identyfikatora zamówienia oraz identyfikatora produktu.

3. **III Postać Normalna (3NF)**: usunięto zależności przechodnie. Nazwa kategorii została wydzielona do osobnej tabeli Kategorie. W tabeli Produkty przechowywany jest jedynie klucz obcy do kategorii. Ponadto zapewniono zapisywanie historycznej ceny w Szczegółach_zamówienia, aby modyfikacja cennika nie fałszowała archiwalnych rachunków.

1.4.4 Schemat notacji IE po normalizacji



4. Definiowanie bazy danych i wprowadzanie danych do bazy

Autorzy: Norbert Antonovitch, Michał Bednarczyk, Jan Balazs de Borbatviz

2.1 4.1. Definicja bazy danych w wariantach PostgreSQL i SQLite

W projekcie zastosowano dwa warianty definicji schematu bazy danych: dla PostgreSQL oraz dla SQLite. Oba warianty odwzorowują tę samą strukturę logiczną, jednak różnią się doбором typów danych i składnią definicji kluczy głównych oraz obcych.

W wariantcie PostgreSQL przyjęto rygorystyczne typowanie danych. Identyfikatory zdefiniowano z użyciem typu SERIAL, pola tekstowe opisano typem VARCHAR o ograniczonej długości, natomiast wartości finansowe zapisano w typie NUMERIC(10,2), zapewniającym odpowiednią precyzję obliczeń.

```
-- Wariant PostgreSQL
CREATE TABLE kategorie (
    id_kategorii SERIAL PRIMARY KEY,
    nazwa VARCHAR(100) NOT NULL
);

CREATE TABLE klienci (
    id_klienta SERIAL PRIMARY KEY,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    telefon VARCHAR(15),
    ulica VARCHAR(255),
    miasto VARCHAR(50),
    kod_pocztowy VARCHAR(15)
);

CREATE TABLE produkty (
    id_produktu SERIAL PRIMARY KEY,
```

(ciąg dalszy na następnej stronie)

(kontynuacja poprzedniej strony)

```

    id_kategorii INTEGER NOT NULL,
    nazwa VARCHAR(255) NOT NULL,
    cena_bazowa NUMERIC(10,2) NOT NULL,
    CONSTRAINT fk_kategoria
        FOREIGN KEY (id_kategorii) REFERENCES kategorie(id_kategorii)
);

CREATE TABLE zamowienia (
    id_zamowienia SERIAL PRIMARY KEY,
    id_klienta INTEGER NOT NULL,
    data_zlozenia TIMESTAMP NOT NULL,
    status VARCHAR(50) NOT NULL,
    CONSTRAINT fk_klient
        FOREIGN KEY (id_klienta) REFERENCES klienci(id_klienta)
);

CREATE TABLE szczegoly_zamowienia (
    id_zamowienia INTEGER NOT NULL,
    id_produktu INTEGER NOT NULL,
    ilosc SMALLINT NOT NULL CHECK (ilosc > 0),
    cena_historyczna NUMERIC(10,2) NOT NULL,
    PRIMARY KEY (id_zamowienia, id_produktu),
    CONSTRAINT fk_zamowienie
        FOREIGN KEY (id_zamowienia) REFERENCES zamowienia(id_zamowienia),
    CONSTRAINT fk_produkt
        FOREIGN KEY (id_produktu) REFERENCES produkty(id_produktu)
);

```

Wariant SQLite został dostosowany do prostszego modelu typów danych oferowanego przez ten silnik. Typ SERIAL zastąpiono konstrukcją INTEGER PRIMARY KEY AUTOINCREMENT, pola tekstowe oraz daty zapisano jako TEXT, natomiast wartości liczbowe o charakterze ciągłym opisano typem FLOAT.

```

-- Wariant SQLite
CREATE TABLE kategorie (
    id_kategorii INTEGER PRIMARY KEY AUTOINCREMENT,
    nazwa TEXT NOT NULL
);

CREATE TABLE klienci (
    id_klienta INTEGER PRIMARY KEY AUTOINCREMENT,
    imie TEXT NOT NULL,
    nazwisko TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    telefon TEXT,
    ulica TEXT,
    miasto TEXT,
    kod_pocztowy TEXT
);

CREATE TABLE produkty (
    id_produktu INTEGER PRIMARY KEY AUTOINCREMENT,
    id_kategorii INTEGER NOT NULL,

```

(ciąg dalszy na następnej stronie)

(kontynuacja poprzedniej strony)

```

    nazwa TEXT NOT NULL,
    cena_bazowa FLOAT NOT NULL,
    FOREIGN KEY (id_kategorii) REFERENCES kategorie(id_kategorii)
);

CREATE TABLE zamowienia (
    id_zamowienia INTEGER PRIMARY KEY AUTOINCREMENT,
    id_klienta INTEGER NOT NULL,
    data_zlozenia TEXT NOT NULL,
    status TEXT NOT NULL,
    FOREIGN KEY (id_klienta) REFERENCES klienci(id_klienta)
);

CREATE TABLE szczegoly_zamowienia (
    id_zamowienia INTEGER NOT NULL,
    id_produktu INTEGER NOT NULL,
    ilosc INTEGER NOT NULL CHECK (ilosc > 0),
    cena_historyczna FLOAT NOT NULL,
    PRIMARY KEY (id_zamowienia, id_produktu),
    FOREIGN KEY (id_zamowienia) REFERENCES zamowienia(id_zamowienia),
    FOREIGN KEY (id_produktu) REFERENCES produkty(id_produktu)
);

```

2.2 4.2. Wybór mechanizmów wsadowego wprowadzania danych

W celu sprawnego zainicjowania bazy danymi testowymi zrezygnowano z ręcznego przygotowywania pojedynczych instrukcji `INSERT` na rzecz automatyzacji realizowanej w języku Python. Dane wejściowe przygotowano w postaci pliku JSON, co umożliwiło jego łatwe przetwarzanie i przenoszenie pomiędzy wariantami bazy.

Do komunikacji z obiema bazami wykorzystano odpowiednie interfejsy programistyczne: moduł `sqlite3` dla SQLite oraz bibliotekę ORM `SQLAlchemy` dla PostgreSQL. Istotnym elementem implementacji było zastosowanie wstawiania wsadowego przy użyciu funkcji `executemany()`. Mechanizm ten pozwala przesyłać wiele rekordów w ramach jednego wywołania, co ogranicza narzut komunikacyjny i skraca czas zasilania bazy danymi.

Zastosowanie podejścia wsadowego ma szczególne znaczenie w przypadku większej liczby rekordów, ponieważ redukuje liczbę pojedynczych operacji na serwerze i pozwala lepiej wykorzystać mechanizm transakcyjny systemu zarządzania bazą danych. W praktyce przekłada się to na większą wydajność niż przy ręcznym wykonywaniu kolejnych poleceń `INSERT`.

2.3 4.3. Komentarz do procesu wprowadzania danych

Proces zasilania bazy danych musiał uwzględniać zależności wynikające z więzów integralności referencyjnej. Z tego względu kolejność importu danych nie mogła być przypadkowa i musiała odzwierciedlać strukturę relacyjną modelu.

Kolejność wprowadzania danych była następująca:

1. W pierwszej kolejności zaimportowano dane do tabel niezależnych, czyli **kategorie** oraz **klienci**.
2. Następnie zasilono tabelę **produkty**. Operacja ta była możliwa dopiero po wcześniejszym utworzeniu kategorii, ponieważ każdy produkt musi być przypisany do istniejącej kategorii.
3. W kolejnym kroku wprowadzono rekordy do tabeli **zamowienia**, przypisując je do istniejących klientów.

4. Na końcu uzupełniono tabelę `szczegoly_zamowienia`, która pełni funkcję encji asocjacyjnej i łączy identyfikatory zamówień oraz produktów. Wymaga to wcześniejszego istnienia obu powiązanych rekordów.

Takie uporządkowanie procesu importu danych wynika bezpośrednio z konstrukcji schematu relacyjnego i stanowi warunek poprawnego zachowania integralności bazy. W praktyce oznacza to, że dane nadrzędne muszą zostać zapisane przed danymi zależnymi.